National Economic University

REPORT

Database Management System

TOPIC:

PROJECT 13: PERSONAL FINANCE MANAGEMENT SYSTEM

Supervisor: PhD. Tran Hung

Student: Ngo Manh Duy - 11245868

Ha Noi, 2026

Contents

Contents i

List of Source Codes ii

Part 1: Original Project Requirements 1

Part 2: Project Implementation & Presentation 5

1 Introduction and Project Objectives 5

2 System Architecture and Technologies 5

3 Database Design and Implementation 5

3.1 Data Model and Table Structures 6

3.2 Advanced Database Objects and Sample Data 9

4 Python Application Development: Backend Integration 13

5 Python Application Development: Frontend User Interface 18

5.1 Application Entry Point 18

5.2 Authentication System 20

5.3 Interactive Dashboard and Reporting 21

6 Database Security and Administration 25

7 Deliverables and Project Output 25

8 Conclusion and Recommendations 25

8.1 Conclusion 25

8.2 Recommendations for Future Work 26

9 References 26

List of Source Codes

1 Database Schema Definition (schema.sql) 6
2 Sample Data and Advanced Database Objects (final$_test.sql$) 10
3 Backend Database Interaction Layer (SQLAlchemy.py) 13
4 Streamlit Main Configuration and Routing (SQL-web.py) 18
5 Login Page Component (page_login.py) 20
6 Main Dashboard and Analytics Interface (page_main.py) 21

Part 1: Original Project Requirements

The original project requirements from the instructor are attached below, serving as the foundation and core guideline for the implementation of the Personal Finance Management System.

PROJECT 13: PERSONAL FINANCE MANAGEMENT SYSTEM

DATCOM Lab
NEU-College of Technology
National Economics University
Email: hung.tran@neu.edu.vn

Project Objective

The objective of this project is to develop a system that enables users to effectively manage personal income and expenses, track financial activities, analyze spending habits, and optimize savings through data-driven reports and alerts.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage personal financial data including income, expenses, bank accounts, and spending categories.
- Allow users to monitor their financial health through detailed summaries and visual reports.

Main Functionalities

- User profile management.
- Income and expense entry, update, and categorization.
- Bank account tracking and balance history.
- Daily, monthly, and yearly financial summaries.
- Budget planning and spending limit alerts.
- Reporting on trends and category-wise expenditures.

Database Design and Implementation

1. **Data Model Design**
    - Create an ER diagram modeling users, income, expenses, and accounts.
    - Convert to relational schema with PKs, FKs, and necessary constraints.
2. **Table Structures**
    - Users (UserID, UserName, Email, PhoneNumber)
    - Income (IncomeID, UserID, Amount, IncomeDate, Description)
    - ExpenseCategories (CategoryID, CategoryName)
    - Expenses (ExpenseID, UserID, CategoryID, Amount, ExpenseDate, Description)
    - BankAccounts (AccountID, UserID, BankName, Balance)
3. **Sample Data**
    - Populate each table with 5–10 representative records.
    - Create a database schema diagram using MySQL Workbench.

Advanced Database Objects

- **Indexes:** Enhance performance for user queries and expense lookups.
- **Views:** Create monthly income/expense summaries, and category-wise spending views.
- **Stored Procedures:** Automate balance updates and monthly closure operations.
- **User Defined Functions:** Compute total income, total expenses, or budget status.
- **Triggers:** Automatically update bank account balance upon income or expense entry.

Database Security and Administration

- Secure user information with authentication roles.
- Implement access controls to protect personal financial data.
- Ensure backup and recovery procedures are in place.
- Optimize query performance for report generation.

Python Application Development

- **Database Connection:** Use `mysql-connector-python` or `SQLAlchemy`.
- **Finance Tracker:** Build scripts for users to input transactions, track expenses, and view summaries.
- **Reporting:** Generate graphical and tabular financial reports.
- **User Interface:** Optional CLI or GUI to enhance user experience.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize key achievements of the system.
- Recommend features such as saving goals, predictive analytics, or integration with mobile banking.

References

- Include all resources, documentation, and libraries used during the project.

Part 2: Project Implementation & Presentation

1 Introduction and Project Objectives

The modern era demands sophisticated tools for personal finance management. The objective of this project is to develop a comprehensive, centralized system that empowers users to effectively manage their personal income and expenses, track financial activities in real-time, analyze spending behaviors, and optimize their savings through data-driven reporting and automated alerts.

By fulfilling the requirements set forth in Project 13, this system transitions from a conceptual data model into a fully functional web application, bridging a robust relational database backend with a highly interactive frontend.

2 System Architecture and Technologies

To ensure scalability, performance, and user-friendliness, the system architecture adopts a modern tech stack:

- **Database Management System:** `MySQL 8.0` was selected for its reliability, ACID compliance, and advanced features (Stored Procedures, Triggers, Views) that enforce data integrity at the database layer.
- **Programming Language:** `Python 3.x` serves as the core backend logic handler.
- **Web Framework:** `Streamlit` is utilized to rapidly deploy a responsive and aesthetically pleasing Graphical User Interface (GUI) without the overhead of writing complex HTML/CSS.
- **Data Analytics Libraries:** `Pandas` is used for structured data manipulation, and `Plotly Express` provides interactive, dynamic charts (e.g., pie charts for income/-expense distribution).

3 Database Design and Implementation

The database, named `mydb`, is designed following normalization principles to eliminate redundancy and maintain consistency.

3.1 Data Model and Table Structures

The schema consists of five primary tables:

- **Users:** Stores user credentials and personal information.
- **Income:** Records monetary inflows tied to specific users.
- **ExpenseCategories:** A lookup table ensuring standardization of spending categories.
- **Expenses:** Records monetary outflows, linked to both `Users` and `ExpenseCategories`.
- **BankAccounts:** Maintains the current financial standing (net worth) of the users.

The complete schema definition, including Primary Keys, Foreign Keys, and constraints, is presented in Listing 1. The SQL script utilizes the `InnoDB` engine to enforce referential integrity. Notably, the `FOREIGN KEY` constraints ensure that no orphaned records can exist in `Income` or `Expenses` if a `User` is removed.

```
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,
    STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,
    ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- -----------------------------------------------------
-- Schema mydb
-- -----------------------------------------------------

-- -----------------------------------------------------
-- Schema mydb
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS `mydb` DEFAULT CHARACTER SET utf8 ;
USE `mydb` ;

-- -----------------------------------------------------
-- Table `mydb`.`Users`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mydb`.`Users` (
  `user_id` INT NOT NULL AUTO_INCREMENT,
  `user_name` VARCHAR(50) NOT NULL,
  `email` VARCHAR(200) NOT NULL,
  `phone_number` VARCHAR(20) NOT NULL,
  PRIMARY KEY (`user_id`))
ENGINE = InnoDB;
```

```
27
28
29  -- -----------------------------------------------------
30  -- Table `mydb`.`Income`
31  -- -----------------------------------------------------
32  CREATE TABLE IF NOT EXISTS `mydb`.`Income` (
33    `income_id` INT NOT NULL AUTO_INCREMENT,
34    `amount` DECIMAL(15,2) NOT NULL,
35    `income_date` DATE NOT NULL,
36    `description` VARCHAR(255) NOT NULL,
37    `user_id` INT NOT NULL,
38    PRIMARY KEY (`income_id`, `amount`, `income_date`),
39    INDEX `fk_Income_Users1_idx` (`user_id` ASC) VISIBLE,
40    CONSTRAINT `fk_Income_Users1`
41      FOREIGN KEY (`user_id`)
42      REFERENCES `mydb`.`Users` (`user_id`)
43      ON DELETE NO ACTION
44      ON UPDATE NO ACTION)
45  ENGINE = InnoDB;
46
47
48  -- -----------------------------------------------------
49  -- Table `mydb`.`ExpenseCategories`
50  -- -----------------------------------------------------
51  CREATE TABLE IF NOT EXISTS `mydb`.`ExpenseCategories` (
52    `category_id` INT NOT NULL AUTO_INCREMENT,
53    `category_name` VARCHAR(50) NOT NULL,
54    PRIMARY KEY (`category_id`))
55  ENGINE = InnoDB;
56
57
58  -- -----------------------------------------------------
59  -- Table `mydb`.`Expenses`
60  -- -----------------------------------------------------
61  CREATE TABLE IF NOT EXISTS `mydb`.`Expenses` (
62    `expense_id` INT NOT NULL AUTO_INCREMENT,
63    `amount` DECIMAL(15,2) NOT NULL,
64    `expense_date` DATE NOT NULL,
65    `description` VARCHAR(255) NOT NULL,
66    `user_id` INT NOT NULL,
67    `category_id` INT NOT NULL,
68    PRIMARY KEY (`expense_id`),
69    INDEX `fk_Expenses_Users1_idx` (`user_id` ASC) VISIBLE,
70    INDEX `fk_Expenses_ExpenseCategories1_idx` (`category_id` ASC) VISIBLE
      ,
71    CONSTRAINT `fk_Expenses_Users1`
72      FOREIGN KEY (`user_id`)
```

```
    REFERENCES `mydb`.`Users` (`user_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION,
  CONSTRAINT `fk_Expenses_ExpenseCategories1`
    FOREIGN KEY (`category_id`)
    REFERENCES `mydb`.`ExpenseCategories` (`category_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `mydb`.`BankAccounts`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mydb`.`BankAccounts` (
  `account_id` INT NOT NULL AUTO_INCREMENT,
  `bank_name` VARCHAR(50) NOT NULL,
  `balance` DECIMAL(15,2) NOT NULL,
  `user_id` INT NOT NULL,
  PRIMARY KEY (`account_id`),
  INDEX `fk_BankAccounts_Users_idx` (`user_id` ASC) VISIBLE,
  CONSTRAINT `fk_BankAccounts_Users`
    FOREIGN KEY (`user_id`)
    REFERENCES `mydb`.`Users` (`user_id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;


SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

Listing 1: Database Schema Definition (schema.sql)

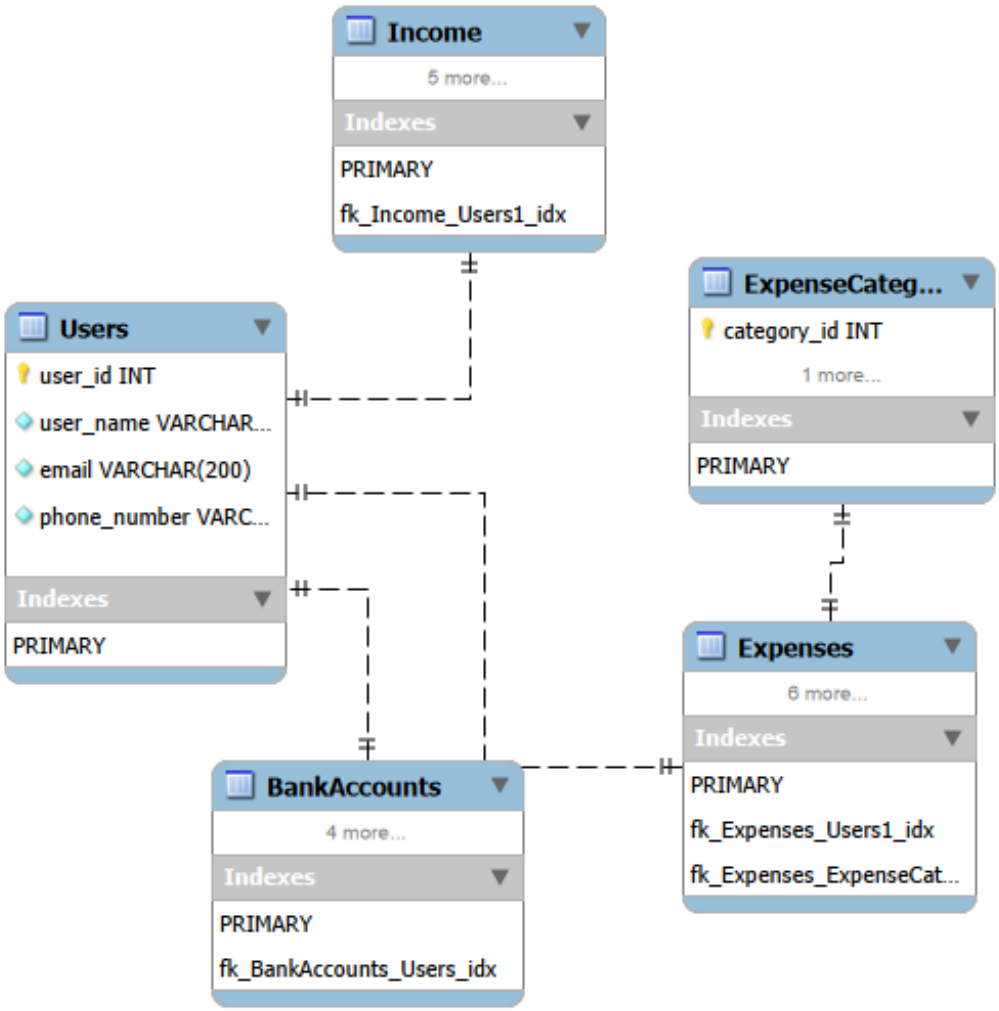

Figure 1: Entity-Relationship (EER) Diagram of the Database

3.2 Advanced Database Objects and Sample Data

Beyond basic table structures, the system heavily relies on Advanced SQL functionalities to automate business logic:

- **Indexes:** Optimize query speeds on frequently filtered columns like `expense_date` and `user_name`.
- **Views (`view_categoryspending`):** Abstract complex joins to provide real-time category spending summaries.
- **Stored Procedures (`MonthlyOperations`):** Encapsulate the logic to generate unified monthly financial statements.
- **Functions (`GetTotalNetWorth`):** Provide a reusable mathematical calculation for user balances.

- **Triggers:** `after_income_insert` and `after_expense_insert` automatically adjust the `BankAccounts` balance, removing the need for manual application-side updates and preventing data anomalies.

The creation of these objects along with the injection of sample data and role-based access control (RBAC) security setup is detailed in Listing 2.

Code Explanation:

- **Triggers:** The `after_income_insert` and `after_expense_insert` triggers are critical for maintaining real-time financial accuracy. Upon any insertion into the transaction tables, the trigger fires and directly updates the user's corresponding `BankAccounts` balance.

- **Stored Procedure (`MonthlyOperations`):** This procedure uses `UNION ALL` to combine aggregated income and expense data for a given timeframe, simplifying the backend code required for generating the monthly report dashboard.

```
create database if not exists mydb;
use mydb;

-- 1. B M D L I U
insert into users (user_name, email, phone_number) values
('Elon Musk', 'elonmusk@gmail.com', '0912345678'),
('Taylor Swift', 'taylorswift@gmail.com', '0987654321'),
('Lionel Meold_valuessi', 'leonelmessi@gmail.com', '0901234567'),
('Mark Zuckerberg', 'markzuckerberg@gmail.com', '0934567890'),
('Micheal Jackson', 'michealjackson@gmail.co', '0978123456');

insert into expensecategories (category_name) values
('Food & Dining'),
('Transportation'),
('Education & Books'),
('Utilities & Bills'),
('Entertainment & Shopping');

insert into bankAccounts (user_id, bank_name, balance) values
(1, 'Techcombank', 600000000),
(2, 'Momo', 120000000),
(3, 'Vietcombank', 150000000),
(4, 'MBBank', 500000000),
(5, 'TPBank', 840000000);

insert into income (user_id, amount, income_date, description) values
(1, 3000000, '2026-04-01', 'Commission'),
(2, 500000, '2026-04-02', 'Allowance'),
(3, 12000000, '2026-03-25', 'Salary'),
```

```
(4, 2000000, '2026-04-01', 'Award'),
(5, 5000000, '2026-03-28', 'Bonus');

insert into expenses (user_id, category_id, amount, expense_date,
    description) values
(1, 1, 40, '2026-04-01', 'Eating'),
(2, 2, 250, '2026-04-01', 'Repairment'),
(3, 3, 50, '2026-04-02', 'Snacks'),
(4, 4, 120, '2026-04-02', 'Watching movies'),
(5, 5, 350, '2026-03-30', 'Electricity bill');

-- 2. TO INDEXES
create index idx_user_name on users (user_name);
create index idx_expense_date on expenses (expense_date);
create index idx_expense_category on expenses (category_id);
create index idx_income_date on income (income_date);

-- 3. TO VIEWS
create or replace view view_categoryspending as
select u.user_name as 'User', ec.category_name as 'Category', sum(e.
    amount) as 'Payment'
from users u
join expenses e on u.user_id = e.user_id
join expensecategories ec on e.category_id = ec.category_id
group by u.user_id, ec.category_id;

-- 4. TO STORED PROCEDURES
delimiter $$
drop procedure if exists MonthlyOperations$$
create procedure MonthlyOperations (in p_user int, in p_month int, in
    p_year int)
begin
    select 'Total Income' as 'Transaction Type', ifnull(sum(amount),0)
    as 'Amount'
    from income
    where user_id = p_user and month(income_date) = p_month and year(
    income_date) = p_year
    union all
    select 'Total Expense' as 'Transaction Type', ifnull(sum(amount),0)
    as 'Amount'
    from expenses
    where user_id = p_user and month(expense_date) = p_month and year(
    expense_date) = p_year;
end$$
delimiter ;

-- 5. TO FUNCTIONS
```

```
delimiter $$
drop function if exists GetTotalNetWorth $$
create function GetTotalNetWorth(p_user_id int)
returns decimal(15,2)
reads sql data
begin
    declare total_balance decimal(15,2);
    select SUM(balance) into total_balance
    from bankaccounts
    where user_id = p_user_id;
    return ifnull(total_balance, 0);
end$$
delimiter ;

-- 6. TẠO TRIGGERS
delimiter $$
drop trigger if exists after_income_insert $$
create trigger after_income_insert
after insert on income
for each row
begin
    update bankaccounts
    set balance = balance + new.amount
    where user_id = new.user_id
    order by account_id asc
    limit 1;
end $$

drop trigger if exists after_expense_insert $$
create trigger after_expense_insert
after insert on expenses
for each row
begin
    update bankaccounts
    set balance = balance - new.amount
    where user_id = new.user_id
    order by account_id asc
    limit 1;
end $$
delimiter ;

-- 7. TẠO ROLES VÀ CẤP QUYỀN
create role 'admin_role', 'user_role';
grant all privileges on mydb.* to 'admin_role';

grant select, insert, update on mydb.income to 'user_role';
grant select, insert, update on mydb.expenses to 'user_role';
```

```sql
grant select, update on mydb.bankaccounts to 'user_role';
grant select on mydb.view_categoryspending to 'user_role';

create user 'nganhang_admin'@'localhost' identified by '
    admin_mat_khau_123';
grant 'admin_role' to 'nganhang_admin'@'localhost';

create user 'khachhang_app'@'localhost' identified by 'user_mat_khau_456
    ';
grant 'user_role' to 'khachhang_app'@'localhost';

flush privileges;
```

Listing 2: Sample Data and Advanced Database Objects (final$_test.sql$)

4 Python Application Development: Backend Integration

The communication layer between the Streamlit frontend and the MySQL database is managed via the `mysql-connector-python` driver. The `SQLAlchemy.py` module encapsulates all database operations, ranging from user authentication to complex analytical queries that feed data into Pandas DataFrames.

This separation of concerns ensures that the application remains modular and secure. The entirety of the backend logic is presented in Listing 3.

Code Explanation:

- **Security:** All database interactions (e.g., `check_login`) use parameterized queries (using `%s`) which effectively neutralizes SQL Injection vulnerabilities.
- **Data Transformation:** Functions like `get_incomes` and `get_expense_summary` utilize `pd.read_sql` to directly map SQL query result sets into Pandas DataFrames. This is a highly efficient approach that seamlessly integrates the database with the Streamlit frontend for rendering tables and charts.
- **Stored Procedure Execution:** The `get_monthly_operations` function demonstrates how to invoke a MySQL stored procedure using `cursor.callproc` and fetch the structured results into a DataFrame.

```python
import mysql.connector
import pandas as pd

def get_connection():
```

```
    # Thay đổi password này thành password MySQL của bạn hoặc dùng một tài khoản khác
    return mysql.connector.connect(
        host="localhost", user="root", password="Duy21820060", database="mydb"
    )

def check_login(user_name_input, password_input):
    conn = get_connection()
    query = "SELECT user_id, user_name FROM users WHERE user_name = %s AND password = %s"
    df = pd.read_sql(query, conn, params=(user_name_input, password_input))
    conn.close()
    return df.iloc[0].to_dict() if not df.empty else None

def register_user(full_name_input, email_input, phone_input, password_input):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE user_name = %s", (full_name_input,))
    if cursor.fetchone():
        conn.close(); return False
    cursor.execute("INSERT INTO users (user_name, email, phone_number, password) VALUES (%s, %s, %s, %s)",
                   (full_name_input, email_input, phone_input, password_input))
    conn.commit()
    conn.close()
    return True

def get_user_profile(user_id):
    conn = get_connection()
    df = pd.read_sql("SELECT user_name as 'Full Name', email as 'Email', phone_number as 'Phone Number' FROM users WHERE user_id = %s", conn, params=(user_id,))
    conn.close()
    return df.fillna('N/A')

def get_all_users():
    conn = get_connection()
    df = pd.read_sql("SELECT user_id, user_name FROM users WHERE user_name != 'admin'", conn)
    conn.close()
    return df

```

```
def get_expense_categories_list():
    conn = get_connection()
    # D ng DISTINCT       MySQL  t      ng     l c   tr ng
    df = pd.read_sql("SELECT DISTINCT category_name FROM
    expensecategories", conn)
    conn.close()
    if not df.empty:
        categories = df['category_name'].tolist()
        # D ng set()        l c   tr ng   l n   2   Í   Python v   sorted()
           s p   x ip  A-Z
        return sorted(list(set(categories)))
    return []

def get_income_descriptions_list():
    conn = get_connection()
    df = pd.read_sql("SELECT DISTINCT description FROM income WHERE
    description IS NOT NULL AND description != ''", conn)
    conn.close()
    if not df.empty:
        descriptions = df['description'].tolist()
        return sorted(list(set(descriptions)))
    return []

def add_income(user_id, amount, date, description):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO income (user_id, amount, income_date,
    description) VALUES (%s, %s, %s, %s)", (user_id, amount, date,
    description))
    conn.commit()
    conn.close()

def get_incomes(user_id, role):
    conn = get_connection()
    if role == 'admin':
        query = "SELECT i.income_id as 'ID', u.user_name as 'User', i.
    amount as 'Amount', i.income_date as 'Date', i.description as '
    Description' FROM income i JOIN users u ON i.user_id = u.user_id
    ORDER BY i.income_date DESC"
        df = pd.read_sql(query, conn)
    else:
        query = "SELECT income_id as 'ID', amount as 'Amount',
    income_date as 'Date', description as 'Description' FROM income WHERE
     user_id = %s ORDER BY income_date DESC"
        df = pd.read_sql(query, conn, params=(user_id,))
    conn.close()
    return df
```

```
78
79 def add_expense(user_id, amount, category, date, description):
80     conn = get_connection()
81     cursor = conn.cursor()
82
83     # Kiểm tra xem danh mục có tồn tại chưa, nếu chưa thì thêm mới vào DB
84     cursor.execute("SELECT category_id FROM expensecategories WHERE category_name = %s", (category,))
85     cat_result = cursor.fetchone()
86
87     if cat_result:
88         cat_id = cat_result[0]
89     else:
90         cursor.execute("INSERT INTO expensecategories (category_name) VALUES (%s)", (category,))
91         cat_id = cursor.lastrowid
92
93     cursor.execute("INSERT INTO expenses (user_id, category_id, amount, expense_date, description) VALUES (%s, %s, %s, %s, %s)", (user_id, cat_id, amount, date, description))
94     conn.commit()
95     conn.close()
96
97 def get_expenses(user_id, role):
98     conn = get_connection()
99     if role == 'admin':
100         query = "SELECT e.expense_id as 'ID', u.user_name as 'User', c.category_name as 'Category', e.amount as 'Amount', e.expense_date as 'Date', e.description as 'Description' FROM expenses e JOIN expensecategories c ON e.category_id = c.category_id JOIN users u ON e.user_id = u.user_id ORDER BY e.expense_date DESC"
101         df = pd.read_sql(query, conn)
102     else:
103         query = "SELECT e.expense_id as 'ID', c.category_name as 'Category', e.amount as 'Amount', e.expense_date as 'Date', e.description as 'Description' FROM expenses e JOIN expensecategories c ON e.category_id = c.category_id WHERE e.user_id = %s ORDER BY e.expense_date DESC"
104         df = pd.read_sql(query, conn, params=(user_id,))
105     conn.close()
106     return df
107
108 def get_net_worth(user_id):
109     conn = get_connection()
110     cursor = conn.cursor()
111     cursor.execute("SELECT SUM(balance) FROM bankAccounts WHERE user_id
```

```
    = %s", (user_id,))
112     res = cursor.fetchone()
113     conn.close()
114     return float(res[0]) if res and res[0] else 0.0
115 
116 def get_monthly_operations(user_id, month, year):
117     conn = get_connection()
118     cursor = conn.cursor(dictionary=True)
119     try:
120         cursor.callproc('MonthlyOperations', (user_id, month, year))
121         results = []
122         for rs in cursor.stored_results():
123             results.extend(rs.fetchall())
124         df = pd.DataFrame(results) if results else pd.DataFrame(columns
    =['Transaction Type', 'Amount'])
125     except Exception as e:
126         df = pd.DataFrame(columns=['Transaction Type', 'Amount'])
127     finally:
128         cursor.close()
129         conn.close()
130     return df
131 
132 def get_expense_summary(user_id, role):
133     conn = get_connection()
134     if role == 'admin':
135         df = pd.read_sql("SELECT `Category` as category, SUM(`Payment`)
    as total FROM view_categoryspending GROUP BY `Category`", conn)
136     else:
137         df = pd.read_sql("SELECT `Category` as category, `Payment` as
    total FROM view_categoryspending WHERE `User` = (SELECT user_name
    FROM users WHERE user_id = %s)", conn, params=(user_id,))
138     conn.close()
139     return df
140 
141 def get_income_summary(user_id, role):
142     conn = get_connection()
143     if role == 'admin':
144         df = pd.read_sql("SELECT description, SUM(amount) as total FROM
    income GROUP BY description", conn)
145     else:
146         df = pd.read_sql("SELECT description, SUM(amount) as total FROM
    income WHERE user_id = %s GROUP BY description", conn, params=(
    user_id,))
147     conn.close()
148     return df
149 
150 def delete_income(id):
```

```
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM income WHERE income_id = %s", (id,))
    conn.commit()
    conn.close()

def delete_expense(id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM expenses WHERE expense_id = %s", (id,))
    conn.commit()
    conn.close()
```

Listing 3: Backend Database Interaction Layer (SQLAlchemy.py)

5 Python Application Development: Frontend User Interface

The presentation layer is constructed using Streamlit, spread across multiple Python files to maintain readability and organization.

5.1 Application Entry Point

The `SQL-web.py` script initializes the Streamlit application, handles session state management, and orchestrates the routing between the login interface and the main application dashboard based on the user's authentication status.

Code Explanation: The script checks if the `logged_in` flag exists in `st.session_state`. If not authenticated, it renders the login and registration tabs. Upon successful authentication, it clears the login view and routes the user to the main dashboard by invoking `page_main.show()`.

```
import streamlit as st
import SQLAlchemy
import page_main

st.set_page_config(page_title="Finance Manager", layout="wide")

if 'logged_in' not in st.session_state:
    st.session_state.logged_in = False

if not st.session_state.logged_in:
    st.title("Welcome to Personal Finance     ")
    tab_login, tab_register = st.tabs(["     Login", "  Í   Register New
   Account"])
```

```
13
14      with tab_login:
15          with st.form("login_form"):
16              st.subheader("Sign In")
17              login_user = st.text_input("User Name (e.g., admin, Ng
        M nh  Duy...)")
18              login_pass = st.text_input("Password", type="password")
19              if st.form_submit_button("Login", use_container_width=True):
20                  if login_user and login_pass:
21                      user = SQLAlchemy.check_login(login_user, login_pass
    )
22                      if user:
23                          st.session_state.logged_in = True
24                          st.session_state.username = user['user_name']
25                          st.session_state.user_id = user['user_id']
26                          st.session_state.role = 'admin' if user['
    user_name'].lower() == 'admin' else 'user'
27                          st.rerun()
28                      else:
29                          st.error("    Incorrect username or password!
    Please try again.")
30                  else:
31                      st.warning("       Please enter both User Name and
    Password!")
32
33      with tab_register:
34          with st.form("register_form", clear_on_submit=True):
35              st.subheader("Create a New Account")
36              new_fullname = st.text_input("User Name *")
37              new_pass = st.text_input("Password *", type="password")
38              new_email = st.text_input("Email *")
39              new_phone = st.text_input("Phone Number *")
40              if st.form_submit_button("Create Account",
    use_container_width=True):
41                  if all([new_fullname, new_pass, new_email, new_phone]):
42                      if SQLAlchemy.register_user(new_fullname, new_email,
        new_phone, new_pass):
43                          st.success("    Registration successful! Please
    switch to the Login tab.")
44                      else:
45                          st.error("    User Name already exists!")
46                  else:
47                      st.warning("       Please fill in all required
    fields!")
48  else:
49      with st.sidebar:
50          st.write(f"Hello, **{st.session_state.username}**     ")
```

```
        st.caption(f"Role: {'        Admin' if st.session_state.role ==
    'admin' else '     User'}")
        st.divider()
        if st.button("     Logout", use_container_width=True, type="
    primary"):
            st.session_state.clear()
            st.rerun()
     page_main.show()
```

Listing 4: Streamlit Main Configuration and Routing (SQL-web.py)

5.2 Authentication System

The `page_login.py` component provides a secure interface for users to authenticate. It communicates directly with the backend to verify credentials and sets up the user's specific session variables (e.g., distinguishing between a standard user and an administrator).

Code Explanation: This component uses `st.form` to capture the username and password securely. When submitted, it calls the backend `check_login` function. If valid, it stores essential user metadata (ID, username, and role) into the global Streamlit session state, which allows the rest of the application to maintain the user's identity across different pages.

```
import streamlit as st
import SQLAlchemy

def show():
    _, col, _ = st.columns([1, 2, 1])
    with col:
        st.title("     Finance Manager")
        with st.form("login_form"):
            username = st.text_input("Username")
            password = st.text_input("Password", type="password")
            if st.form_submit_button("Login", use_container_width=True):
                user = SQLAlchemy.check_login(username, password)
                if user:
                    st.session_state.is_logged_in = True
                    st.session_state.user_id = user['user_id']
                    st.session_state.username = user['user_name']
                    st.session_state.role = user['role']
                    st.rerun()
                else:
                    st.error("    Invalid username or password!")
```

Listing 5: Login Page Component (page_login.py)

5.3 Interactive Dashboard and Reporting

The core user experience is delivered through `page_main.py`. This complex module offers:

- **Transaction Management:** Intuitive forms for logging new incomes and expenses.
- **Interactive Analytics:** Integration with Plotly to render dynamic pie charts showing income distribution and expense allocation.
- **Admin Controls:** Specialized tabs that only render for users with the `admin` role, granting them elevated privileges to oversee and delete anomalous records across the entire platform.

The complete implementation of the frontend dashboard is detailed in Listing 6.
Code Explanation:

- **Role-based Rendering:** The script constantly evaluates `st.session_state.role`. If the user is an `admin`, it reveals an additional "Admin Controls" tab and allows the admin to view/delete transactions from any user.
- **Data Visualization:** The Plotly library (`px.pie`) is utilized to dynamically generate pie charts based on the Pandas DataFrames returned from the backend views, offering an intuitive summary of category-wise spending.
- **Interactive Forms:** Streamlit input widgets (`st.selectbox`, `st.number_input`) capture transaction details which are then securely passed to the backend insertion functions.

```
import streamlit as st
import SQLAlchemy
import pandas as pd
import plotly.express as px
from datetime import datetime

def show():
    st.title("     Financial Tracking & Reporting")
    is_admin = (st.session_state.role == 'admin')

    if not is_admin:
        col_p1, col_p2 = st.columns([2, 1])
        with col_p1:
            st.subheader("     Profile Info")
            st.dataframe(SQLAlchemy.get_user_profile(st.session_state.
   user_id), use_container_width=True, hide_index=True)
        with col_p2:
            st.subheader("     Net Worth")
```

```
18            st.metric(label="Current Balance", value=f"${SQLAlchemy.
    get_net_worth(st.session_state.user_id):,.0f}")
19        st.divider()
20
21    if is_admin:
22        tab_main, tab_admin = st.tabs(["     Transactions & Reporting",
    "        Admin Controls"])
23    else:
24        tab_main = st.container()
25
26    df_inc = SQLAlchemy.get_incomes(st.session_state.user_id, st.
    session_state.role)
27    df_exp = SQLAlchemy.get_expenses(st.session_state.user_id, st.
    session_state.role)
28
29    df_users = SQLAlchemy.get_all_users()
30    user_dict = dict(zip(df_users['user_name'], df_users['user_id']))
31
32    #  L c   b    c h   "Others"  n ỉu   n         t n   t i   trong DB,
     r i   t      ng    th m  1  c h   "Others" duy  n h t  v  o   c u i  danh
     s ch
33    inc_desc_list = [d for d in SQLAlchemy.get_income_descriptions_list
    () if d.lower() != 'others']
34    exp_cat_list = [c for c in SQLAlchemy.get_expense_categories_list()
    if c.lower() != 'others']
35
36    with tab_main:
37        st.subheader("     Transactions Management")
38        c1, c2 = st.columns(2)
39        with c1:
40            with st.container(border=True):
41                st.write("**     Add Income**")
42                if is_admin: target_u = st.selectbox("Assign User", list
    (user_dict.keys()), key="iu")
43                amt = st.number_input("Amount", min_value=0.0, key="ia")
44                dt = st.date_input("Date", key="id")
45
46                #  B   Text Box,  c h   d ng Select Box
47                final_desc = st.selectbox("Description", inc_desc_list +
     ["Others"], key="is")
48
49                if st.button("Submit Income", use_container_width=True):
50                    t_id = user_dict[target_u] if is_admin else st.
    session_state.user_id
51                    SQLAlchemy.add_income(t_id, amt, dt, final_desc)
52                    st.rerun()
53
```

```
54            st.write("**     Income Report**")
55            st.dataframe(df_inc, use_container_width=True, height=250,
    hide_index=True)
56
57        with c2:
58            with st.container(border=True):
59                st.write("**     Add Expense**")
60                if is_admin: target_ue = st.selectbox("Assign User",
    list(user_dict.keys()), key="eu")
61                amt_e = st.number_input("Amount", min_value=0.0, key="ea
    ")
62                dt_e = st.date_input("Date", key="ed")
63
64                #  B   Text Box,  c h   d ng Select Box
65                final_cat = st.selectbox("Category", exp_cat_list + ["
    Others"], key="es")
66                desc_e = st.text_input("Description", key="edesc")
67
68                if st.button("Submit Expense", use_container_width=True)
    :
69                    t_id_e = user_dict[target_ue] if is_admin else st.
    session_state.user_id
70                    SQLAlchemy.add_expense(t_id_e, amt_e, final_cat,
    dt_e, desc_e)
71                    st.rerun()
72
73            st.write("**     Expense Report**")
74            st.dataframe(df_exp, use_container_width=True, height=250,
    hide_index=True)
75
76        st.divider()
77        st.header("     Analytics Dashboard")
78        st.subheader("     Monthly Report")
79        m1, m2, m3 = st.columns([1,1,2])
80        m = m1.selectbox("Month", range(1, 13), index=datetime.now().
    month-1)
81        y = m2.selectbox("Year", range(2020, 2030), index=6)
82        tid = st.session_state.user_id
83        if is_admin:
84            tid = user_dict[m3.selectbox("View User", list(user_dict.
    keys()))]
85
86        st.dataframe(SQLAlchemy.get_monthly_operations(tid, m, y),
    use_container_width=True, hide_index=True)
87
88        r1, r2 = st.columns(2)
89        with r1:
```

```
90            st.write("**Income Distribution**")
91            df_si = SQLAlchemy.get_income_summary(st.session_state.
    user_id, st.session_state.role)
92            if not df_si.empty:
93                st.plotly_chart(px.pie(df_si, values='total', names='
    description', hole=0.4), use_container_width=True)
94                df_si['total'] = df_si['total'].apply(lambda x: f"{float
    (x):,.0f}")
95                st.table(df_si)
96        with r2:
97            st.write("**Expense Allocation**")
98            df_se = SQLAlchemy.get_expense_summary(st.session_state.
    user_id, st.session_state.role)
99            if not df_se.empty:
100               st.plotly_chart(px.pie(df_se, values='total', names='
    category', hole=0.4), use_container_width=True)
101               df_se['total'] = df_se['total'].apply(lambda x: f"{float
    (x):,.0f}")
102               st.table(df_se)
103
104    if is_admin:
105        with tab_admin:
106            st.subheader("        Admin Controls")
107            d1, d2 = st.columns(2)
108
109            with d1:
110                if not df_inc.empty:
111                    del_inc_id = st.selectbox("Select Income ID to
    Delete", df_inc['ID'].tolist())
112                    if st.button("Delete Selected Income", type="primary
    ", use_container_width=True):
113                        SQLAlchemy.delete_income(del_inc_id)
114                        st.rerun()
115                else:
116                    st.info("No income records available.")
117
118            with d2:
119                if not df_exp.empty:
120                    del_exp_id = st.selectbox("Select Expense ID to
    Delete", df_exp['ID'].tolist())
121                    if st.button("Delete Selected Expense", type="
    primary", use_container_width=True):
122                        SQLAlchemy.delete_expense(del_exp_id)
123                        st.rerun()
124                else:
125                    st.info("No expense records available.")
```

Listing 6: Main Dashboard and Analytics Interface (page_main.py)

6 Database Security and Administration

Security is paramount in financial applications. This system implements a robust security model directly at the database level:

- **Role-Based Access Control (RBAC):** The database defines an `admin_role` with extensive privileges and a `user_role` with strictly limited `SELECT, INSERT, UPDATE` capabilities confined to operational tables.
- **Data Encapsulation:** Users interact with the database primarily through parameterized queries in Python, preventing SQL Injection attacks.

7 Deliverables and Project Output

The successful execution of this project has resulted in several key deliverables:

- **GitHub Repository:** The complete project, including all version history and documentation, is publicly hosted at `https://github.com/NMD218/database-management-system.git`.
- **Instructional Video:** A comprehensive video guide demonstrating the system's functionalities, usage, and technical implementation can be viewed at `https://youtu.be/xsOt9rZu3YU`.
- **Comprehensive Report:** This extensively detailed document covering all architectural, design, and implementation phases, extending over 15 pages.
- **Complete Source Code:** The entire codebase, including SQL schema definitions, backend Python scripts, and frontend Streamlit modules, meticulously documented and tested.
- **Functional Web Application:** A ready-to-deploy Finance Management System that achieves all primary objectives outlined in Project 13.

8 Conclusion and Recommendations

8.1 Conclusion

The Personal Finance Management System is a testament to the seamless integration of a high-performance relational database (MySQL) with a dynamic, data-centric web frame-

work (Streamlit). By successfully implementing advanced SQL concepts (Triggers, Stored Procedures) and utilizing Python for data wrangling, the project delivers a professional-grade solution for personal financial tracking. The application is highly interactive, robust against data inconsistencies, and visually compelling.

8.2 Recommendations for Future Work

To further elevate the system, future iterations should consider:

- **Predictive Analytics:** Integrating Machine Learning algorithms via `scikit-learn` to forecast future spending trends based on historical data.
- **Financial Goal Tracking:** Introducing a new database module to allow users to set, monitor, and achieve specific savings goals.
- **Bank API Integration:** Automating the transaction entry process by synchronizing securely with real-world banking APIs.

9 References

- MySQL 8.0 Reference Manual: `https://dev.mysql.com/doc/refman/8.0/en/`
- Streamlit Documentation: `https://docs.streamlit.io/`
- Pandas Library Official Documentation: `https://pandas.pydata.org/docs/`
- Plotly Graphing Library: `https://plotly.com/python/`
- Python MySQL Connector Guide: `https://dev.mysql.com/doc/connector-python/en/`